

EQTMFUSION v0.1.0

eQTMFusion

An eQTM-Guided Multi-Omics Machine Learning Framework for
Predicting Asthma Severity, Lung Function, IgE, and Other Clinical
Outcomes

[User Manual](#) · [Installation](#) · [API Reference](#) · [CLI Reference](#) · [Troubleshooting](#)

Table of Contents

- 1. Introduction & Scope
- 2. Installation Guide
- 3. Data Formatting Guide
- 4. Quick Start Tutorial
- 5. Advanced Tutorial: Custom Cohorts & Outcomes
- 6. CLI Reference
- 7. Publication-Quality Outputs (Figures & Tables)
- 8. Python API Reference
 - 8.1 simulation · 8.2 qc · 8.3 preprocessing · 8.4 eqtm · 8.5 feature_selection · 8.6 fusion · 8.7 models · 8.8 explainability · 8.9 visualization · 8.10 reporting · 8.11 utilities
- 9. Troubleshooting Guide
- 10. Best Practices Guide
- 11. Implementation Status & Known Limitations

1. Introduction & Scope

eQTMFusion is a Python package for integrating multiple omics modalities (DNA methylation, gene expression, SNPs, miRNA, metabolomics, proteomics, and any custom modality you provide) to predict clinical outcomes, with a particular focus on asthma severity, lung function (FEV1), and IgE. Its central methodological idea is **eQTM-guided feature selection**: methylation features are prioritized using expression quantitative trait methylation (eQTM) associations discovered in your own cohort, rather than selecting features by variance alone.

The package is disease-agnostic. The asthma framing (clinical variables, synthetic data generator) is a worked example; the underlying pipeline — QC, preprocessing, eQTM analysis, tiered feature selection, multi-omics fusion, prediction, explainability, and reporting — applies to any cohort with the same tabular structure.

Honesty note: this manual documents what is actually implemented and tested in the shipped package (49/49 unit tests passing, including subprocess-level CLI tests). Section 11 lists items that are intentionally out of scope or only partially implemented, so nothing here is oversold.

1.1 Package Structure

```
eqtmfusion/  
├─ simulation/          synthetic multi-omics asthma cohort generator  
├─ qc/                 missingness / outlier QC  
├─ preprocessing/      imputation, scaling, categorical/ordinal encoding  
├─ eqtm/               cis-/trans-eQTM analysis (OLS + mixed models, FDR)  
├─ feature_selection/  Tier1 (eQTM prior) / Tier2 (variance, MAD) /  
│                     Tier3 (LASSO, ElasticNet, RF, XGBoost, Boruta, SHAP)  
├─ fusion/             early / intermediate (AE, VAE) / late (ensemble,  
│                     stacking) / graph-based fusion  
├─ models/             classification, ordinal (CORAL, ordinal logistic),  
│                     regression  
├─ explainability/     SHAP, permutation importance, integrated gradients, LIME  
├─ visualization/     ROC/PR, volcano, heatmap, PCA/UMAP/t-SNE, correlation,  
│                     feature importance, eQTM network  
├─ reporting/          Jinja2 HTML report (+ optional PDF via weasyprint)  
├─ utilities/          metrics, CV / bootstrap / external validation, logging  
└─ cli/               click-based command-line interface  
  
tests/                 49 unit tests (pytest)  
examples/              run_end_to_end.py: full working walkthrough  
docs/                 this manual
```

2. Installation Guide

2.1 Requirements

Python 3.10 or later. Core dependencies (numpy, pandas, scipy, scikit-learn, torch, statsmodels, matplotlib, jinja2, click) install automatically. Optional extras add xgboost, lightgbm, catboost, shap, lime, umap-learn, seaborn, networkx, Boruta, and weasyprint (for PDF reports).

2.2 Install from source

```
unzip eqTMFusion.zip
cd eqTMFusion

# Core install
pip install -e .

# Full feature set (recommended)
pip install -e ".[full]"

# Development tools (pytest, black, ruff, mypy, pre-commit)
pip install -e ".[dev]"
```

2.3 Docker

```
docker build -t eqtmfusion .
docker run --rm eqtmfusion simulate --out /data --n-subjects 200
docker run --rm -v $(pwd)/data:/data eqtmfusion run-all \
  --data-dir /data --outcome severity --out /data/results
```

2.4 Conda

```
conda env create -f environment.yml
conda activate eqtmfusion
```

2.5 Verifying the install

```
eqtmfusion --help
pytest tests/ -q      # should report "30 passed"
```

PDF report generation via weasyprint has system-level dependencies (Pango, Cairo, GDK-PixBuf) beyond the Python package. If `pip install weasyprint` succeeds but PDF generation fails, see Section 9 (Troubleshooting) or consult https://doc.courtbouillon.org/weasyprint/stable/first_steps.html for platform-specific system libraries.

3. Data Formatting Guide

eQTMFusion expects a directory containing one `clinical.csv` file plus any number of omics CSVs. There is no fixed list of required omics — every CSV file other than `clinical.csv` is automatically treated as a separate modality, named after its filename.

3.1 clinical.csv

First column (or index) must be `subject_id`. Any other columns are treated as clinical variables and can be used as an outcome. The built-in simulator produces these columns as a reference schema:

```
subject_id, age, sex, bmi, race, smoking_status, asthma_status, severity,
gina_step, fev1, fvc, fev1_fvc, ige, eosinophils, ics_use,
hospitalization, exacerbation_count
```

You are not restricted to these column names — any column can be passed to `--outcome` on the CLI or to the Python API.

3.2 Omics CSVs (any modality)

Each omics file must be a *subjects* × *features* matrix with `subject_id` as the first column/index, matching the same subject IDs used in `clinical.csv` (partial overlap across modalities is fine — QC and fusion handle missing/partial subjects).

```
data/
├─ clinical.csv
├─ methylation.csv      -> modality "methylation"
├─ expression.csv      -> modality "expression"
├─ snps.csv            -> modality "snps"
├─ mirna.csv           -> modality "mirna"
├─ metabolomics.csv    -> modality "metabolomics"
├─ proteomics.csv       -> modality "proteomics"
└─ my_custom_omic.csv  -> modality "my_custom_omic" (works automatically)
```

3.3 eQTM prior (optional but recommended)

If a `ground_truth.json` file is present in the data directory with a `causal_cpgs` key (a list of CpG IDs), those CpGs are used as the Tier 1 biological prior for methylation feature selection. In real usage, this list should come from running `eqtmfusion.eqtm.run_eqtm_analysis` on your own cohort first (see Section 5), not from a simulator.

4. Quick Start Tutorial

4.1 Generate a demo cohort

```
eqtmfusion simulate --out data/ \  
  --n-subjects 300 --n-cpgs 2000 --n-genes 1000 \  
  --n-snps 500 --n-mirna 100 --n-metabolites 100 --n-proteins 200
```

4.2 Run the full pipeline

```
# Ordinal severity prediction (auto-discovers all 6 omics files, auto-infers  
# task type from the outcome column, runs 5-fold CV by default)  
eqtmfusion run-all --data-dir data/ --outcome severity --out results/  
  
# Binary classification  
eqtmfusion run-all --data-dir data/ --outcome asthma_status --out results/  
  
# Continuous regression  
eqtmfusion run-all --data-dir data/ --outcome fev1 --out results/
```

4.3 View the report

Open [results/report.html](#) in a browser. It includes data summary tables, QC results, feature selection breakdown (Tier1 eQTM-prioritized vs. Tier2 variance-backfilled counts per modality), and both in-sample and cross-validated metrics side by side.

4.4 Convert the report to PDF

```
python3 -c "  
from eqtmfusion.reporting import generate_pdf_report  
generate_pdf_report('results/report.html', 'results/report.pdf')  
"
```

5. Advanced Tutorial: Custom Cohorts & Outcomes

5.1 Using your own data

Replace the simulated CSVs with your own, keeping the same column conventions (Section 3). No code changes are required — the CLI and API auto-discover modalities from whatever CSVs are present.

5.2 Custom outcome columns

```
# Any clinical.csv column works, not just the built-in ones
eqtmfusion run-all --data-dir data/ --outcome exacerbation_count \
  --task regression --model xgboost --out results/ --n-cv-folds 5
```

`--task auto` (the default) infers classification vs. ordinal vs. regression from the column's dtype and number of unique values. Override with `--task classification|ordinal|regression` when the inference would be ambiguous (e.g. a low-cardinality count variable).

5.3 Running the eQTM analysis yourself

```
from eqtmfusion.eqtm import run_eqtm_analysis, get_significant_eqtm_cpgs
import pandas as pd

methylation = pd.read_csv("data/methylation.csv", index_col=0)
expression = pd.read_csv("data/expression.csv", index_col=0)
clinical = pd.read_csv("data/clinical.csv", index_col=0)
covariates = clinical[["age", "sex", "bmi", "smoking_status"]]

result = run_eqtm_analysis(
    methylation, expression, covariates,
    candidate_pairs=None,          # None = all CpG x gene combinations
                                # (use a pre-filtered candidate list for
                                # genome-wide data; see Section 9.3)
    eqtm_type="cis", model="linear", fdr_alpha=0.05,
)
sig_cpgs = get_significant_eqtm_cpgs(result, fdr_thresh=0.05)
```

5.4 Building a Python pipeline manually

```
from eqtmfusion.qc import run_qc
from eqtmfusion.preprocessing import preprocess_omics_matrix
from eqtmfusion.feature_selection import combine_tier1_tier2
from eqtmfusion.fusion import early_fusion
from eqtmfusion.models import get_classifier
from eqtmfusion.explainability import shap_explain
from eqtmfusion.visualization import plot_roc_curve
from eqtmfusion.reporting import generate_html_report

methylation, meth_qc = run_qc(methylation, "methylation")
methylation = preprocess_omics_matrix(methylation)
features, report = combine_tier1_tier2(methylation, sig_cpgs, target_n=500)
methylation = methylation[features]

fused = early_fusion({"methylation": methylation, "expression": expression})
clf = get_classifier("xgboost")
clf.fit(fused.values, y.values)

shap_df = shap_explain(clf, fused.values, fused.columns.tolist(), model_type="tree")
```

See [examples/run_end_to_end.py](#) in the package for a complete, verified-working script covering simulate → QC → preprocess → eQTM → feature-select → fuse → train → explain → visualize → report.

6. CLI Reference

6.1 `eqtmfusion simulate`

Option	Default	Description
--out	data	Output directory
--n-subjects	200	Number of subjects
--n-cpgs	2000	Number of CpGs
--n-genes	1000	Number of genes
--n-snps	500	Number of SNPs
--n-mirna	100	Number of miRNAs
--n-metabolites	100	Number of metabolites
--n-proteins	200	Number of proteins
--seed	42	Random seed

6.2 `eqtmfusion run-all`

Option	Default	Description
--data-dir	required	Directory with clinical.csv + any omics CSVs
--outcome	severity	Any column name in clinical.csv
--task	auto	auto classification ordinal regression
--model	(task default)	random_forest, xgboost, lightgbm, catboost, logreg, mlp (classification); elasticnet, xgboost, lightgbm, mlp (regression); CORAL always used for ordinal
--out	results	Output directory for report.html
--target-n-features	500	Features retained per modality (Tier1+Tier2)
--eqtm-modality	methylation	Which discovered modality gets the eQTM Tier1 prior
--n-cv-folds	5	K-fold CV for reportable metrics; 0 = in-sample only (fast, exploratory)
--device	auto	auto cpu cuda (for the CORAL ordinal model)

7. Publication-Quality Outputs (Figures & Tables)

Every `run-all` invocation automatically writes a full publication-ready output bundle to `<out>/figures/` and `<out>/tables/`, styled per Nature-family journal conventions: sans-serif fonts, thin axis lines, no top/right spines, outward ticks, the Okabe-Ito colorblind-safe palette, lettered panels (A, B, C...), and 300+ dpi PNG plus vector PDF export for every figure.

7.1 Figures

File	Contents
Figure1_model_diagnostics	Task-dependent 4-panel diagnostic: classification (ROC, PR, confusion matrix, calibration curve); regression (predicted-vs-observed, residuals-vs-fitted, residual histogram, Bland-Altman agreement plot); ordinal (row-normalized confusion matrix, per-class accuracy). Computed on out-of-fold cross-validated predictions , not in-sample, whenever <code>--n-cv-folds > 0</code> .
Figure2_feature_importance	Horizontal bar chart of top features by cross-validated permutation importance, colored by omics modality.
Figure3_cv_performance_summary	Box + strip plot of each metric's distribution across CV folds, for reporting model stability rather than a single point estimate.
Figure4_eqtm_forest_plot	Forest plot ($\beta \pm 95\%$ CI) of the top eQTM associations, generated automatically when the demo simulator's <code>ground_truth.json</code> is present; for real cohorts, generate this yourself from <code>run_eqtm_analysis</code> output and pass it to <code>generate_publication_outputs(eqtm_results=...)</code> .

7.2 Tables

File	Contents
Table1_cohort_characteristics	Conventional clinical Table 1: continuous variables as mean (SD), categorical as n (%), stratified by outcome group with between-group test p-values (Welch's t-test / ANOVA / chi-square as appropriate).
Table2_model_performance_summary	Mean $\pm$ SD across CV folds for every reported metric, formatted as publication-ready strings.
Table3_top_biomarkers	Ranked, modality-annotated feature importance table (Modality / Feature / importance value columns).

Critical methodology note: feature importance is computed via `cv_permutation_importance` (retrain-per-fold, evaluate on held-out data), not naive in-sample permutation importance. With hundreds of features and modest sample sizes, in-sample permutation importance on an overfit model collapses to ~ 0 for every feature, because permuting one column barely perturbs a decision surface that already memorized the training labels via other feature combinations. This was found and fixed during development — see Section 11.4.

7.3 Generating the bundle manually

```
from eqtmfusion.reporting import generate_publication_outputs

outputs = generate_publication_outputs(
    outdir="results/", task="classification",
    y_true=oof_y_true, y_pred=oof_y_pred, y_proba=oof_y_proba, # out-of-fold!
    clinical_df=clinical, group_col="asthma_status",
    cv_metrics_df=cv_metrics_df,
    importance_df=importance_df, importance_value_col="importance_mean",
    eqtm_results=eqtm_result.results, # optional
    class_labels=["Control", "Case"],
    title_prefix="Asthma cohort: ",
)
```

Any input left as `None` is skipped rather than raising an error, so this can be called with partial results (e.g. no eQTM analysis run, or no clinical dataframe for Table 1).

7.4 Individual figure/table functions

For custom layouts, call the individual functions directly: `eqtmfusion.visualization.fig_classification_diagnostics`, `fig_regression_diagnostics`, `fig_ordinal_diagnostics`, `fig_feature_importance_panel`, `fig_cv_performance_summary`, `fig_eqtm_forest_plot`; and `eqtmfusion.reporting.table1_cohort_characteristics`, `table_model_performance_summary`, `table_top_biomarkers`, `save_table` (CSV/XLSX/LaTeX), and `dataframe_to_image` (render any table as a publication-style PNG, e.g. for a supplementary "Table S1" panel). Use `eqtmfusion.visualization.save_publication_figure` to export any custom matplotlib figure at 300+ dpi in PNG/PDF/SVG together.

8. Python API Reference

8.1 `eqtmfusion.simulation`

`simulate_cohort()`, `save_cohort()`, `SimulatedCohort`, `SCHEMA`

Generates a synthetic multi-omics cohort with embedded ground-truth eQTM pairs and causal features, for testing and demonstration.

`SCHEMA` documents the exact clinical column names expected elsewhere in the package.

8.2 `eqtmfusion.qc`

`run_qc()`, `missingness_summary()`, `detect_outlier_samples_iqr()`, `QCReport`

Drops features/samples exceeding missingness thresholds and flags outlier samples via an IQR rule on per-sample feature means.

8.3 `eqtmfusion.preprocessing`

`impute_missing()`, `scale_features()`, `preprocess_omics_matrix()`, `encode_clinical_categoricals()`, `encode_severity_ordinal()`

kNN or mean/median imputation, standard scaling, and categorical/ordinal encoding for clinical variables.

8.4 `eqtmfusion.eqtm`

`run_eqtm_analysis()`, `get_significant_eqtm_cpgs()`, `get_significant_eqtm_genes()`, `EQTMResult`

cis-/trans-eQTM testing via OLS or linear mixed models, with covariate adjustment and Benjamini-Hochberg FDR correction. Tests are run per candidate (CpG, gene) pair; for genome-wide unrestricted scans, pre-filter your candidate list (see Section 9.3).

8.5 `eqtmfusion.feature_selection`

`tier1_eqtm_prior()`, `tier2_variance_threshold()`, `tier2_mad_ranking()`, `combine_tier1_tier2()`, `tier3_lasso_selection()`, `tier3_elasticnet_selection()`, `tier3_random_forest_importance()`, `tier3_xgboost_importance()`, `tier3_boruta_selection()`, `tier3_shap_selection()`

Three-tier selection: Tier 1 (eQTM biological prior), Tier 2 (variance/MAD, unsupervised backfill), Tier 3 (supervised ML-based, applied against a specific outcome).

8.6 `eqtmfusion.fusion`

`early_fusion()`, `intermediate_fusion_autoencoder()`, `late_fusion_ensemble()`, `late_fusion_stacking()`, `graph_fusion_embed()`, `Autoencoder`, `VariationalAutoencoder`, `SimpleGCNFusion`

Four fusion strategies. Graph-based fusion is a lightweight single-layer GCN reference implementation — see Section 11 for its limitations relative to a full attention-based GAT.

8.7 `eqtmfusion.models`

`get_classifier()`, `get_regressor()`, `CoralOrdinalNet`, `train_coral_model()`, `predict_coral()`, `fit_ordinal_logistic()`, `predict_ordinal_logistic()`

Classifiers: logreg, random_forest, xgboost, lightgbm, catboost, mlp. Regressors: elasticnet, xgboost, lightgbm, mlp. Ordinal: CORAL neural network (rank-consistent) or proportional-odds ordinal logistic regression.

8.8 eqtmfusion.explainability

shap_explain(), permutation_importance_explain(), integrated_gradients(), lime_explain(), top_biomarkers_by_modality()

SHAP (tree or kernel), permutation importance, integrated gradients (for any PyTorch model in this package), and an optional LIME wrapper. `top_biomarkers_by_modality()` splits a fused-feature importance table back out per modality (CpG/gene/SNP/miRNA/protein/metabolite).

8.9 eqtmfusion.visualization

plot_roc_curve(), plot_pr_curve(), plot_volcano(), plot_heatmap(), plot_pca(), plot_embedding(), plot_correlation_matrix(), plot_feature_importance(), plot_eqtm_network()

`plot_embedding()` uses UMAP if `umap-learn` is installed, otherwise falls back to t-SNE automatically.

8.10 eqtmfusion.reporting

generate_html_report(), generate_pdf_report()

Jinja2-based HTML report with embedded base64 figures; optional PDF export via weasyprint.

8.11 eqtmfusion.utilities

classification_metrics(), regression_metrics(), ordinal_metrics(), concordance_correlation_coefficient(), run_cv(), bootstrap_metric(), external_validation(), get_logger()

Includes MCC, sensitivity/specificity, CCC (Lin's concordance), and quadratic weighted kappa, alongside standard accuracy/AUC/RMSE/R².

9. Troubleshooting Guide

9.1 `ImportError` for `xgboost` / `lightgbm` / `catboost` / `shap` / `lime` / `boruta` / `umap-learn`

These are optional extras. Install with `pip install -e ".[full]"`, or individually, e.g. `pip install shap`. The package raises a clear `ImportError` naming the missing package rather than failing silently.

9.2 PDF report generation fails

weasyprint requires system libraries (Pango, Cairo, GDK-PixBuf) in addition to the Python package. On Debian/Ubuntu:

```
sudo apt-get install libpango-1.0-0 libpangocairo-1.0-0 libgdk-pixbuf2.0-0
```

On macOS via Homebrew: `brew install pango`. See https://doc.courtbouillon.org/weasyprint/stable/first_steps.html for the full platform matrix. The HTML report remains fully usable even if PDF export is unavailable.

9.3 eQTM analysis is slow or runs out of memory

`run_eqtm_analysis` fits one OLS/mixed model per candidate (CpG, gene) pair. Passing `candidate_pairs=None` tests every combination present in both matrices, which is computationally infeasible for genome-wide data (e.g. 50,000 CpGs × 10,000 genes = 5×10⁸ tests). Always pre-filter to a manageable candidate list: nearby gene-CpG pairs for a cis scan, or a variance/MAD-filtered feature subset for an exploratory trans scan. A warning is raised if your candidate list exceeds `max_pairs_warning` (default 2,000,000).

9.4 LightGBM prints excessive training logs

Fixed in this release — `get_classifier("lightgbm")` and `get_regressor("lightgbm")` set `verbose=-1` by default. If you still see per-iteration logs, you are likely constructing the model directly via `lightgbm.LGBMClassifier(...)` rather than through `eqtmfusion.models.get_classifier`.

9.5 Task auto-inference picks the "wrong" type for my outcome

`--task auto` uses simple heuristics (string/low-cardinality → classification/ordinal, else regression). For genuinely ambiguous columns (e.g. a 0–5 count variable), specify `--task` explicitly rather than relying on inference.

9.6 CORAL ordinal model metrics look unstable across folds

The CORAL network trains from a random initialization each fold with a fixed `n_epochs` (default 200). For small cohorts, either increase epochs, reduce the feature count first (Section 5.4), or fall back to `fit_ordinal_logistic` (proportional-odds regression), which is more stable at small sample sizes at the cost of the linearity assumption.

9.7 `pip install -e .` fails on `--break-system-packages` environments

On externally-managed Python installations (common on recent Debian/Ubuntu), use a virtual environment instead:

```
python3 -m venv .venv  
source .venv/bin/activate  
pip install -e ".[full]"
```

10. Best Practices Guide

10.1 Always report cross-validated metrics, not in-sample

The CLI's `report.html` shows both in-sample and CV metrics side by side deliberately: with hundreds of features and modest sample sizes, in-sample accuracy can reach 1.0 while CV accuracy is far lower. Use `--n-cv-folds 5` (or 10) for any result you intend to report or publish; treat `--n-cv-folds 0` as an exploratory-only mode.

10.2 Derive the eQTM prior from your own cohort, not a generic list

Tier 1 feature selection is only as good as the eQTM analysis behind it. Run `run_eqtm_analysis` on your actual methylation + expression + covariates before setting `--eqtm-modality`, rather than reusing CpG lists from an unrelated cohort or tissue type.

10.3 Match feature-selection budget to sample size

With n subjects, a feature budget well above n (e.g. 500 features for 150 subjects) will overfit badly in-sample. Use the CV metrics to tune `--target-n-features` down until CV performance stabilizes rather than trusting in-sample numbers.

10.4 Prefer Tier 1 + Tier 2 for exploratory work, add Tier 3 for a final model

Tier 3 (LASSO/RF/XGBoost/SHAP) selection is supervised and outcome-specific — running it before finalizing your outcome definition risks selecting features that won't generalize to a revised outcome. Use Tier 1+2 for exploration and integration, and apply Tier 3 once the outcome and cohort are finalized.

10.5 Use early fusion as your baseline before trying AE/VAE/graph fusion

Early fusion (simple concatenation) is the cheapest to compute and the easiest to debug. Establish a baseline with it before moving to intermediate (autoencoder) or graph-based fusion, so you can tell whether the more complex method is actually adding value on your data.

10.6 Treat the graph fusion module as a starting point, not a finished GNN

`fusion.graph_fusion_embed` is a minimal single-layer GCN. For research-grade graph fusion (attention weighting, partial-overlap per-modality graphs), see the limitations noted in Section 11.2.

11. Implementation Status & Known Limitations

11.1 Fully implemented and tested

Component	Status
Synthetic multi-omics simulator	Tested
QC, preprocessing, imputation, scaling	Tested
cis-/trans-eQTM analysis (OLS + mixed models, FDR)	Tested
Tier1/2/3 feature selection (incl. SHAP, Boruta)	Tested
Early / intermediate (AE, VAE) / late fusion	Tested
Classification, ordinal (CORAL), regression models	Tested
SHAP, permutation importance, integrated gradients, LIME	Tested
Visualization suite (incl. UMAP, seaborn, networkx paths)	Tested
Publication-quality figures (6 multi-panel types, Nature styling, 300+ dpi)	Tested (0 text-overlap collisions across all 6 types)
Publication-standard tables (Table 1, performance summary, biomarkers)	Tested
Cross-validated permutation importance	Tested (regression test proves it does not collapse to zero, unlike the in-sample version)
HTML + PDF reporting	Tested
CLI: auto modality discovery, flexible outcomes, CV metrics, GPU flag, publication bundle	Tested (subprocess-level, 49/49 unit tests passing)

11.2 Intentionally lightweight or partial

Component	Status	Detail
Graph-based fusion	Partial	Single-layer GCN reference implementation, not a full attention-based GAT with per-modality partial-overlap graphs.
"DeepCORAL"	Naming note	The CORAL ordinal network supports arbitrary depth via <code>hidden_dims</code> . True DeepCORAL (Sun & Saenko domain adaptation) is a different method and is not implemented.
eQTM genome-wide scan	Partial	Per-pair OLS, not a vectorized MatrixEQTL-style implementation. Fine for pre-filtered candidate lists; not for unrestricted millions-of-pairs scans.

11.3 Not implemented

Component	Status	Alternative
TabTransformer	Not implemented	Raises <code>NotImplementedError</code> . Use mlp, xgboost, or lightgbm.
Transformer regressor	Not implemented	Raises <code>NotImplementedError</code> . Use mlp, xgboost, or lightgbm.
Sphinx documentation site	Not generated	This manual plus module docstrings.
8 Jupyter notebooks	Not generated	<code>examples/run_end_to_end.py</code> covers the same ground.

11.4 Changelog of bugs found and fixed during verification

1. SHAP 3D-array output format (modern shap versions) crashed both `shap_explain` and `tier3_shap_selection` — fixed. 2. Simulator crashed when `n_eqtm_pairs` exceeded available CpGs/genes on small demo cohorts — now clips with a warning. 3. LightGBM's default verbose training output was extremely noisy — suppressed. 4. pandas 3.0's new default string dtype broke `dtype == object` checks in the CLI's task auto-inference, causing a crash on binary categorical outcomes — fixed with proper dtype checks. 5. The CLI originally only accepted two hardcoded omics filenames and four hardcoded outcome columns — rewritten to auto-discover any CSV as a modality and accept any clinical column as an outcome. 6. **In-sample permutation importance is meaningless for overfit models, and the CLI was doing exactly that:** with hundreds of features and modest sample sizes, the classifier reached 100% training accuracy, and permuting any single feature on that same training data barely moved an already-memorized fit — verified directly, importance was exactly 0.000 for every one of 500 features. Fixed by adding `cv_permutation_importance` , which retrains per CV fold and evaluates on the held-out fold; a regression test directly demonstrates the in-sample version collapsing while the CV version does not, on the same deliberately-overfit synthetic data. 7. `openpyxl` and `Pillow` were installed in the development environment but not declared in `pyproject.toml` / `requirements.txt` , which would have broken a clean install — now declared explicitly.